ECE 1155: Information Security

Cryptographic Hash Functions

Dr. Mai Abdelhakim, PhD

Electrical & Computer Engineering

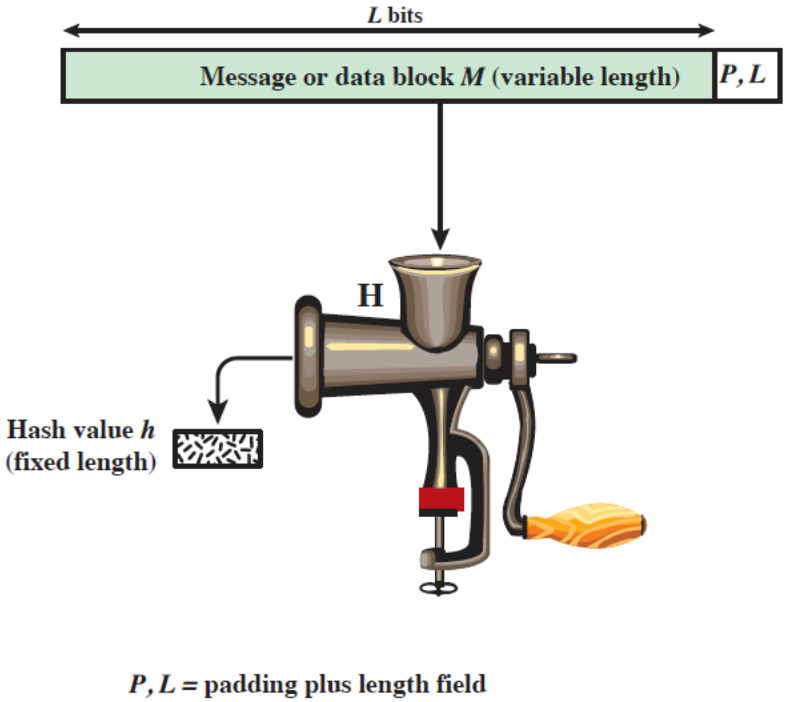
Swanson School of Engineering

University of Pittsburgh

Usage of Hash

- Storing passwords, message and passwords authentication
- Integrity preservation
- Other applications, e.g. Blockchain

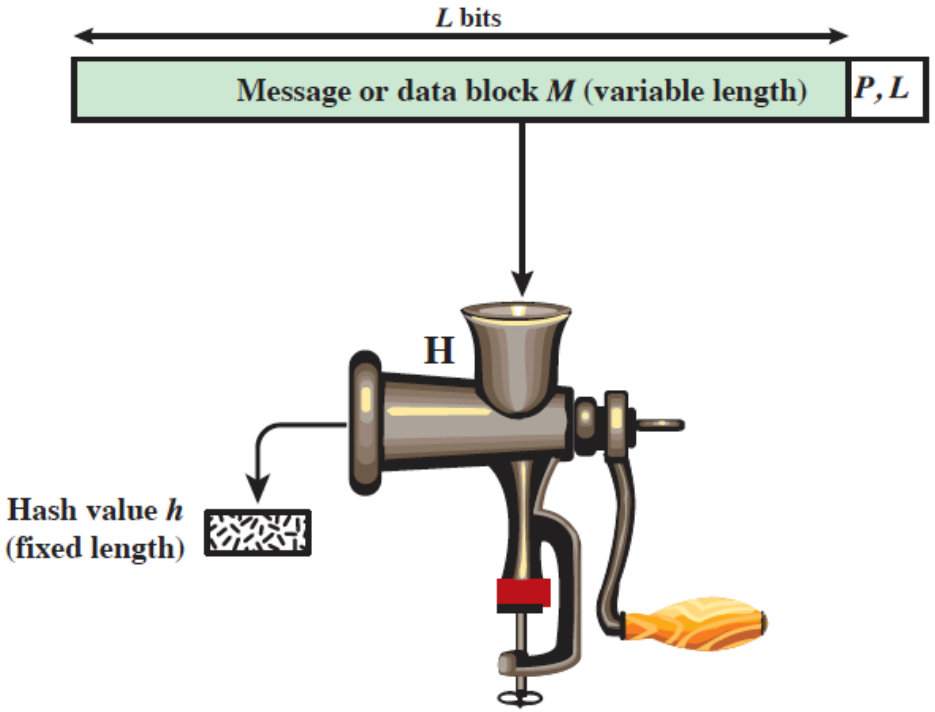
Hash functions properties



- **Hash function:** maps **input data (m) of any size** to **a fixed size** output (H(m), called **hash value** or **message digest**)
  - E.g: Output size 128, 256, 512 bits
- For practical application of the hash, it should also be **easy to compute**
- Techniques that are popular: MD, SHA

Properties of cryptographic hash functions

1. Applied to block of data of any size
2. Produce fixed length output
3. Easy to compute
4. One-way property
5. Collision resistant property



P, L = padding plus length field

Properties of hash functions for integrity verification – One-way function

- **One-way Hash function:** Given H(m) , it is difficult to find m
  - Otherwise attacker sniffing packets, from the hash, can know the message

Properties of hash functions for integrity verification – Collision resistant

- **Collision resistant:**
  - <u>Weak collision resistant:</u> Given m_1 it is computationally infeasible to find $m_2 \neq m_1$ such that $H(m_2) = H(m_1)$
    - Specific to that message m1
  - <u>Strong collision resistant</u>: it is computationally infeasible to find two different messages ($m_2 \neq m_1$) where $H(m_1) = H(m_2)$
- Essential for **integrity validation**

Recall that it is often the case when digital signature is applied to hash instead of entire message

The CA's identity (Symantec)

The owner of the certificate (paypal)

Public key

CA's signature

```
Certificate:
Data:
  Serial Number:
            2c:d1:95:10:54:37:d0:de:4a:39:20:05:6a:f6:c2:7f
  Signature Algorithm: sha256WithRSAEncryption
  Issuer: C=US, O=Symantec Corporation, OU=Symantec Trust Network,
          CN=Symantec Class 3 EV SSL CA - G3
  Validity
     Not Before: Feb  2 00:00:00 2016 GMT
     Not After : Oct 30 23:59:59 2017 GMT
  Subject: 1.3.6.1.4.1.311.60.2.1.3=US/
           1.3.6.1.4.1.311.60.2.1.2=Delaware/
           businessCategory=Private Organization/
           serialNumber=3014267, C=US/
           postalCode=95131-2021, ST=California,
           L=San Jose/street=2211 N 1st St,
           O=PayPal, Inc., OU=CDN Support, CN=www.paypal.com
  Subject Public Key Info:
      Public Key Algorithm: rsaEncryption
         Public-Key: (2048 bit)
         Modulus:
           00:da:43:c8:b3:a6:33:5d:83:c0:63:14:47:fd:6b:22:bd:
           bf:4e:a7:43:11:55:eb:20:8b:e4:61:13:ee:de:fe:c6:e2:
           ... (omitted) ...
           7a:15:00:c5:01:69:b5:10:16:a5:85:f8:fd:07:84:9a:c9:
         Exponent: 65537 (0x10001)
Signature Algorithm: sha256WithRSAEncryption
4b:a9:64:20:cc:77:0b:30:ab:69:50:d3:7f:de:dc:7c:e2:fb:93:84:fd:
78:a7:06:e8:14:03:99:c0:e4:4a:ef:c3:5d:15:2a:81:a1:b9:ff:dc:3a:
... (omitted) ...
fb:00:3e:7d:6a:de:cb:9f:ff:ef:8c:65:35:e4:22:b5:88:b2:48:32:1e:
```

Ref. W. Du

Example: if collision resistance not satisfied

- Forging public-key certificates
  - Assume two certificate requests for www.example.com and www.attacker.com have same hash due to collision
  - CA signing of either would be equivalent
  - Attacker can get certificate signed for www.example.com without owning it

Example: if collision resistance not satisfied

- **Alice purchase from Bob (recall transactions are signed)**
- Bob generates two **invoice** that have the same hash value, one with large $ amount and another with low $ amount
- Bob sends an invoice with low $ to Allice, Alice signs it (validate the purchase)
- Bob uses the large $ amount instead to transfer (has a valid hash and signature to validate it)

  - HASH should have strong collision resistant property

IOU: I Owe you

Example: Hash for software download

VirtualBox

search...
Login Preferences

About
Screenshots
Downloads
Documentation
End-user docs
Technical docs
Contribute
Community

Download VirtualBox

Here you will find links to VirtualBox binaries and its source code.

VirtualBox binaries

By downloading, you agree to the terms and conditions of the respective license.

If you're looking for the latest VirtualBox 5.2 packages, see VirtualBox 5.2 builds. Please also use version 5.2 if you still need support for 32-bit hosts, as this has been discontinued in 6.0. Version 5.2 will remain supported until July 2020.

VirtualBox 6.0.12 platform packages

- Windows hosts
- OS X hosts
- Linux distributions
- Solaris hosts

The binaries are released under the terms of the GPL version 2.

See the changelog for what has changed.

You might want to compare the checksums to verify the integrity of downloaded packages. *The SHA256 checksums should be favored as the MD5 algorithm must be treated as insecure!*

- SHA256 checksums, MD5 checksums

Note: After upgrading VirtualBox it is recommended to upgrade the guest additions as well.

HASH of software

Ref. W. Du

Security Impact of Collision Attacks

- Integrity of Programs
  - Ask trusted entity to sign a legitimate program's hash
  - Attacker creates a malicious program with same hash
  - The signature for legitimate program is also valid for malicious version

Ref. W. Du

Security Impact of Collision Attacks

- These hash collision examples are theoretical with questionable feasibility!
  - We'll analyze it in more details

Revisit: Properties of cryptographic hash functions

1. Applied to block of data of any size

2. Produce fixed length output

3. Relatively easy to compute

4. One-way property
   - there is no feasible way to invert the hash

5. Collision resistant property (strong --and weak-- collision resistance)

Example

- Hash h= x mod 1000. Is that a valid cryptographic hash?
- Check properties:
  - maps arbitrary size data to data of fixed size
  - Easy to compute
  - One-way property
    - For any value h1, we can find set of numbers that produce such hash (h1, 1000 + h1,2000+h1**).. It should be difficult to find any such messages for a given h**.
  - Collision resistant property
    - We can find two numbers that produce same hash value. For example 1005, 2005
      - Collision resistant not satisfied

Hash collisions

- Assume hash of length L bits. Is hash collision impossible?
- How many messages should the attacker try?

Hash function weaknesses -- **Birthday Paradox**

- **How many people you need in a room to have greater than 50% chance that two people will have the same birthday?**

  - Number of days in a year is our container : 365 days
    - Assume birthday is uniform distribution

    - **We might expect the answer is 365+1.. But it is not!!**

Hash function weaknesses -- **Birthday Paradox**

Hash function weaknesses -- **Birthday Paradox**

- Number of people in a room = n, number of days = k (rep. 365 days)

- Number of possibilities we have = k^n
  - Each of the n people can have any of the k birthdays

- Number of possibilities that none will have the same birthday (P(n,r))
  $(K)_n = k\ (k-1)\ (k-2)\ \ldots.\ (k-n+1)$
  - First person has all the k options, next is k-1 options ….

- Probability that there is at least two people sharing the same birthday is:

$$\mathbf{p = 1 - (K)_n / k^n}$$

  - If we set p=0.5, and solve for n we get n almost equals to $\sqrt{k}$

Birthday Paradox

- If n = $\sqrt{k} = \sqrt{365} = 19$ the we get

  p = 1- $(K)_n / k^n \approx n(n-1)/2k \approx 0.5$ ➔ *n= 19 people in the room, there are 50% chance to have two with same birthday*

Referred as Paradox and at first, it does seem paradoxical that only $\sqrt{k}$ people in the room, we'd expect two or more people have same birthday

Conclusion from the birthday paradox

- Length of the hash value (L) is less than length of message
  - That suggests that in principle many input can result in same Hash
  - **From birthday paradox:**
    If length of hash is L , then **k =2^L possible hashes** and if we search over **n= $\sqrt{k}$ = $2^{L/2}$** possible input messages, we expect to find a hash collision (two messages with same hash)
    - Computationally infeasible if L is large
      - Use L at least 128 bits

Tophat question: P4_hash_bdayParadox

Based on the birthday paradox

A secure hash that generates an L bits output can be broken with a brute force work factor of about $2^{L/2}$.

Intuition of the Birthday Paradox

- With n messages, we have order of n^2 pairs of message to compare
- When n= $2^{L/2}$ messages, then there are around 2^L pairs (which is also the total number of possible hashes)
  - If L is the number of bits in hash, then in 2^L comparison we expect a collision
    - What is the number of hashes you need to compute to find **10 collisions**?
      - Sqrt(10 2^L)

Example: birthday paradox attack & Impact of hash collisions

- Alice signs hash of message M , h(M) which is L bits
- Eve can conduct attack:
  - Eve wants Alice to sign "evil" message E, (for example this message confirms that Alice agrees to give $1M to Eve)
  - Eve creates a lot of innocent messages I, the Alice won't mind to sign
    - Eve generates variants of the innocence message (minor edits, with same meaning as I)
  - Eve generates variants of the evil message (minor edits, with same meaning as E)
  - Eve can find one of the variants of evil message (Ei) with same hash as one of the variants of the innocent message (Ii)
  - Eve sends Ii to Alice, Alice signs the Hash. Then Eve, uses the Hash to validate the evil message (Ei)

Ref. Stamp.

Common hash function in security

- Popular cryptographic hash algorithms MD, SHA series

Common Hash function in security – MD series

- **Message digest** (MD) series -- Developed by Ron Rivest (MIT)
  - Includes MD2, MD4, MD5, MD6
    - MD2, MD4 - severely broken (obsolete)
    - MD5 - collision resistance property broken; one-way property not broken
    - MD6 - developed in response to proposal by NIST

Common hash function in security – SHA (secure hash algorithm) series

Developed by National Institute of Standards and Technology (NIST), 1993

- Includes SHA-0, SHA-1, SHA-2, and SHA-3

Number after SHA is version number

  - SHA-0: withdrawn due to **flaw**
  - SHA-1: Designed by NSA; **Collision attack** found in 2017, **160 bits hash**
  - SHA-2: Designed by NSA; Includes **SHA-256, SHA-384 and SHA-512**
    No significant attack found yet
  - SHA-3: Has different construction structure (compared to SHA-1 and SHA-2), Released in 2015

National Security Agency : NSA

Truncated versions of SHA-512

Processing is block by block

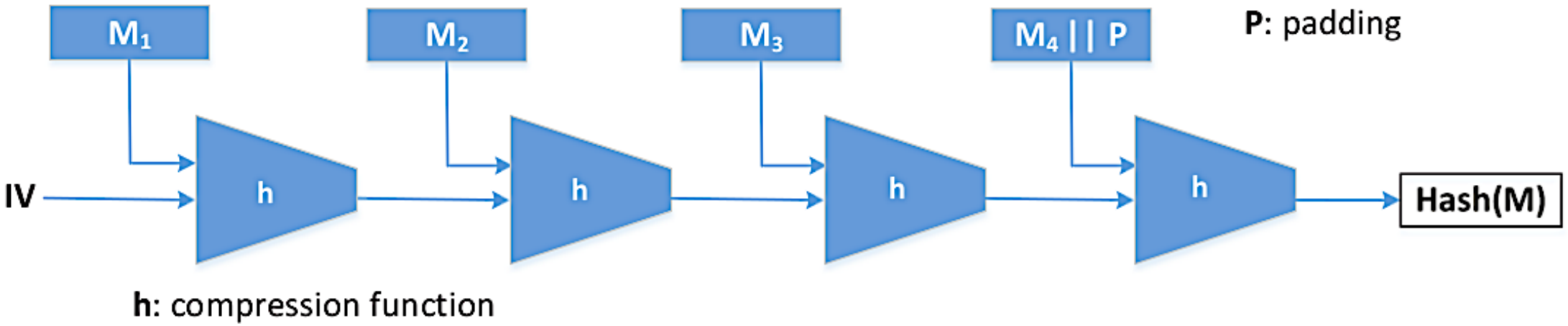
	SHA-1	SHA-224	SHA-256	SHA-384	SHA-512	SHA-512/224	SHA-512/256
Message size	$< 2^{64}$	$< 2^{64}$	$< 2^{64}$	$< 2^{128}$	$< 2^{128}$	$< 2^{128}$	$< 2^{128}$
	Message size can be ignored as it is very large (can be regarded as no limit)						
Word size (bits)	32	32	32	64	64	64	64
Block size (bits)	512	512	512	1024	1024	1024	1024
Message digest size (bits)	160	224	256	384	512	224	256
Number of steps (rounds)	80	64	64	80	80	80	80
Security	80	112	128	192	256	112	128

Notes:

1. All sizes are measured in bits.
2. Security refers to the fact that a birthday attack on a message digest of size L produces a collision with a work factor of approximately $2^{L/2}$.

Basic structure for computing hash functions

Construction used by algorithms like MD, SHA-1, and SHA-2



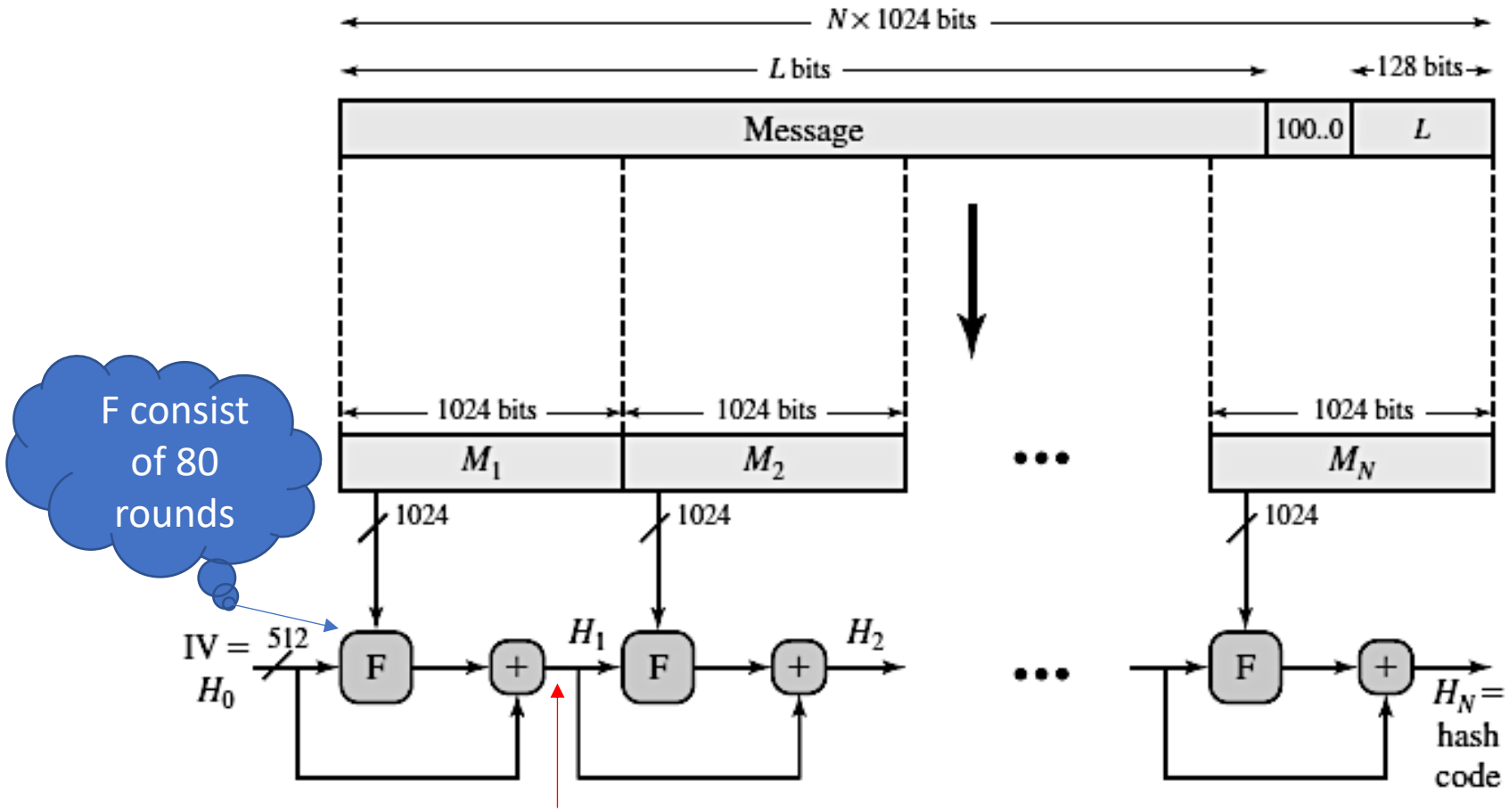
Merkle–Damgard construction method

Ref.: W. Du

Hash generation for SHA 512 : processing of all message

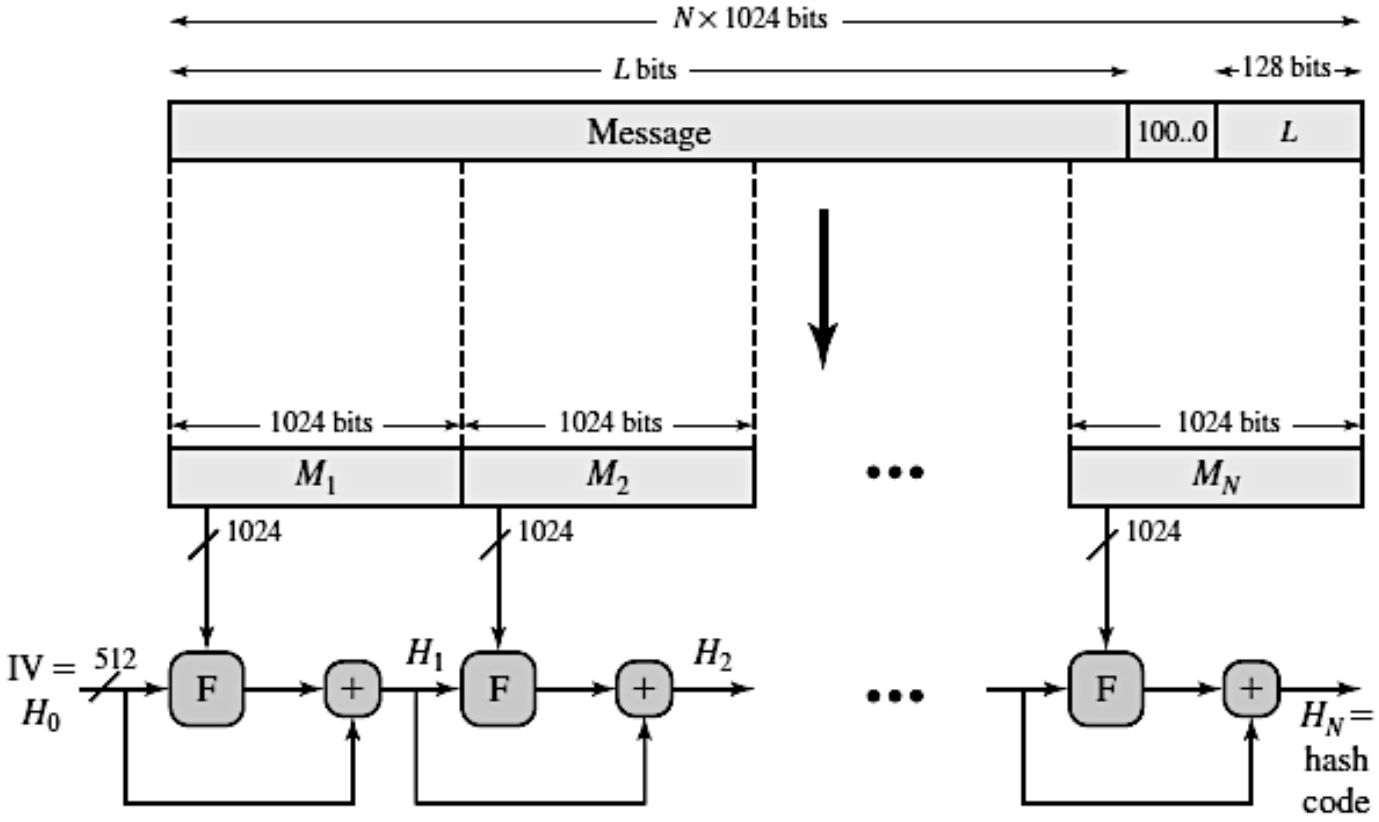
The repetition of the basic function F produces results that are well mixed! it is unlikely that two messages will have the same hash code

L here is input length not hash length

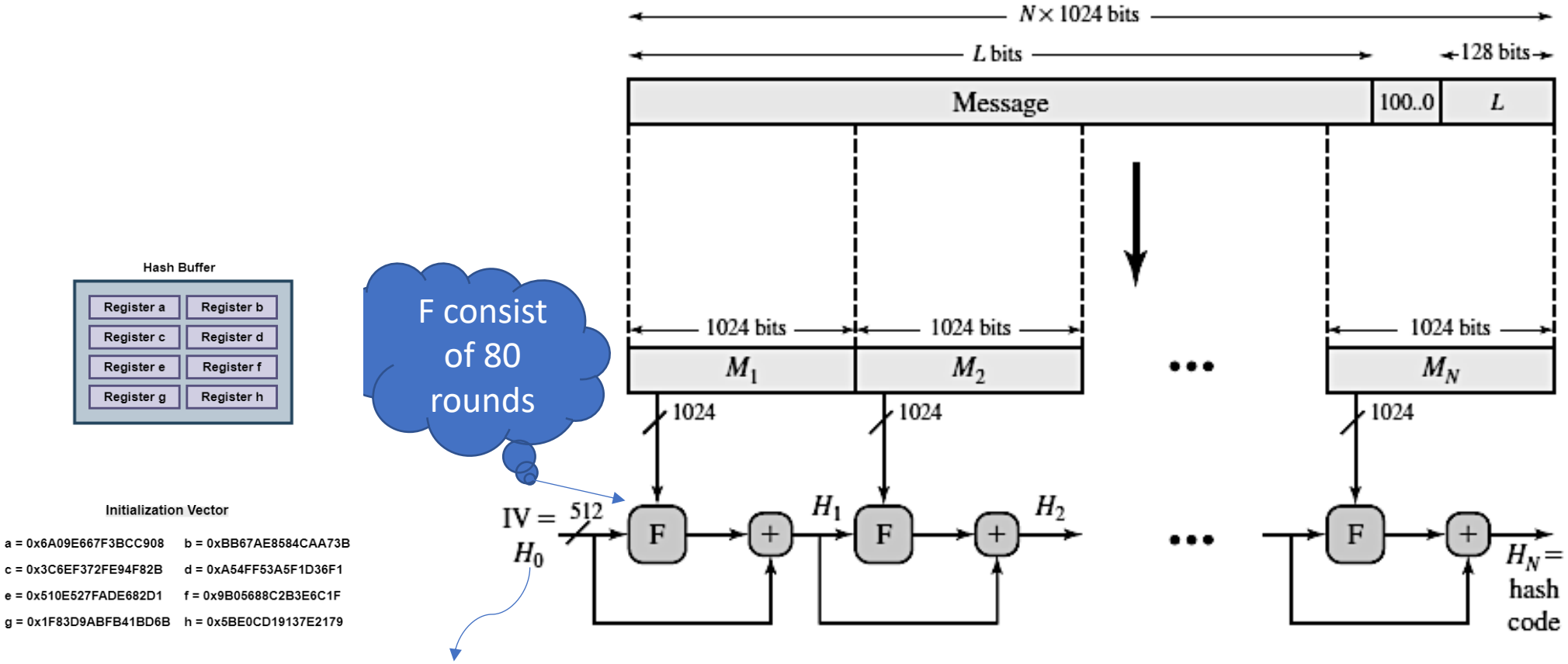


Steps to produce the hash

- **Step 1: Append padding bits**: message size + length (128 bits) must be integer multiple of block size (in SHA 512, block size is 1024)
  - The padding consists of a single 1-bit followed by the necessary number of 0-bits
  - After padding (and before appending 128 bits) the length ≡ 896 (mod 1024)
- **Step 2: Append length:** 128-bit length of the original message before padding

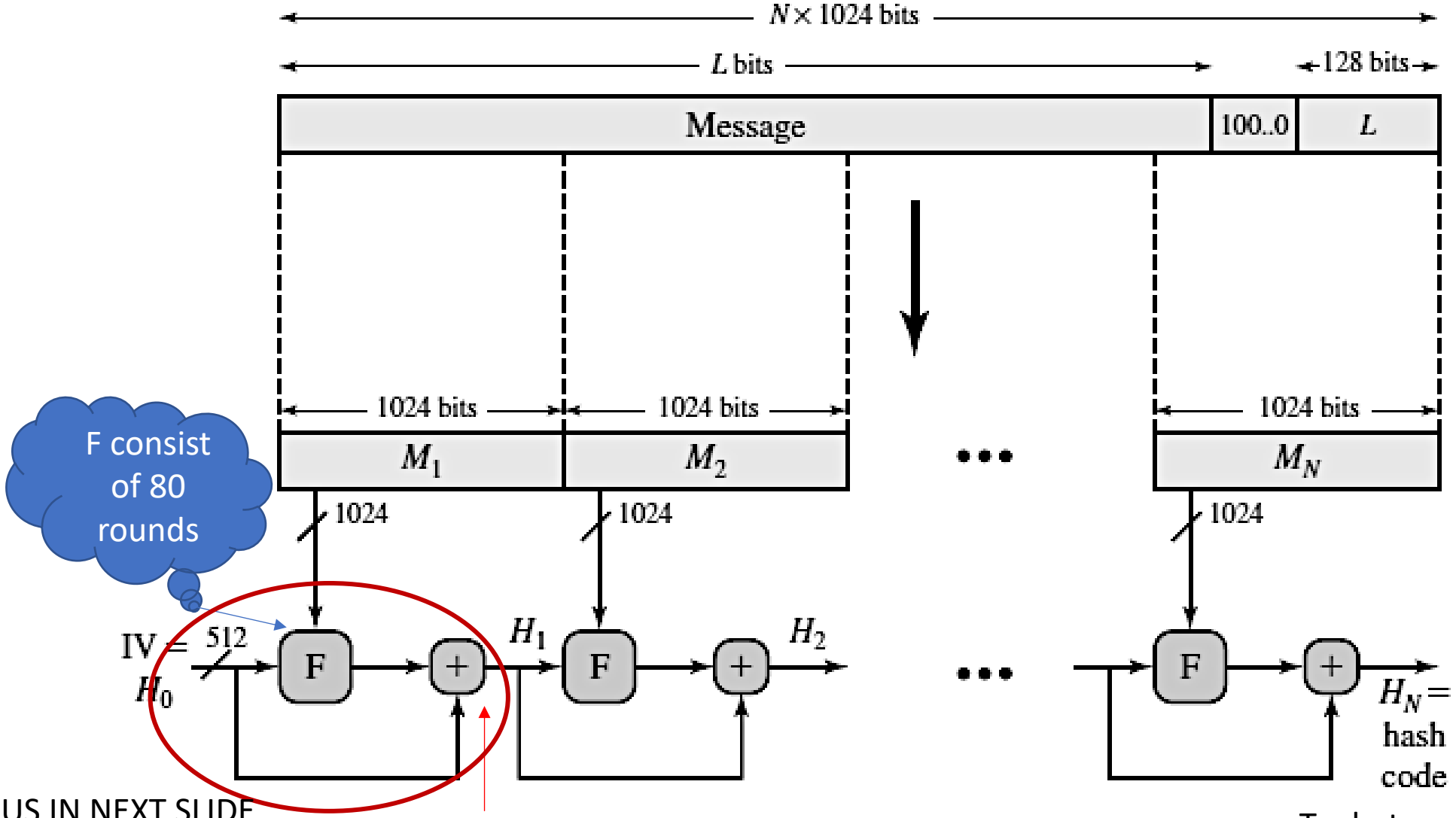


Steps to produce the hash - cont.



- **Step 3: Initialize hash buffer:** A 512-bit buffer is used to hold intermediate and results of the hash function. The buffer can be represented as **<u>eight 64-bit registers (a, b, c, d, e, f, g, h).</u>**
- **Step 4: Process the message in 1024-bit blocks**, <u>The heart of the algorithm is a module that consists of 80 rounds;</u> this module is labeled F in figure
- **Step 5: Output.** After all N 1024-bit blocks have been processed, the output from the Nth stage is the 512-bit message digest.

Hash generation for SHA 512 : processing of all message



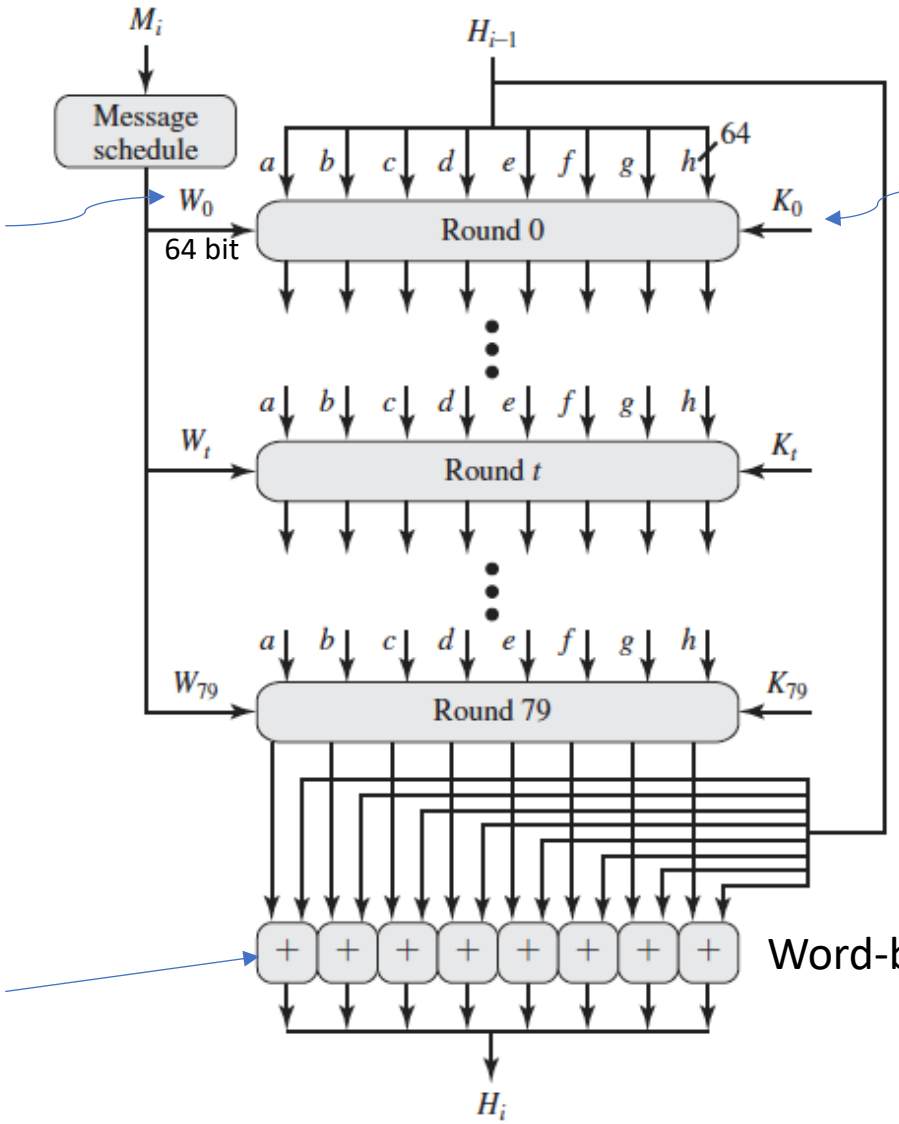
FOCUS IN NEXT SLIDE

Word-by-word addition mod 2^{64}

Tophat question: P4_hash_SHA

Hash generation for SHA 512 : processing of one block of input message

- Each **round** *t:* has W_t and **additive constant** K_t,
  - W_t a 64-bit value, **derived** from the current 1024-bit block being processed **(M_i)**
- The operations performed during a **round** consist of **circular shifts**, and primitive **Boolean functions** based on AND, OR, NOT, and XOR.
- Lastly, word-by-word addition mod 2^{64}



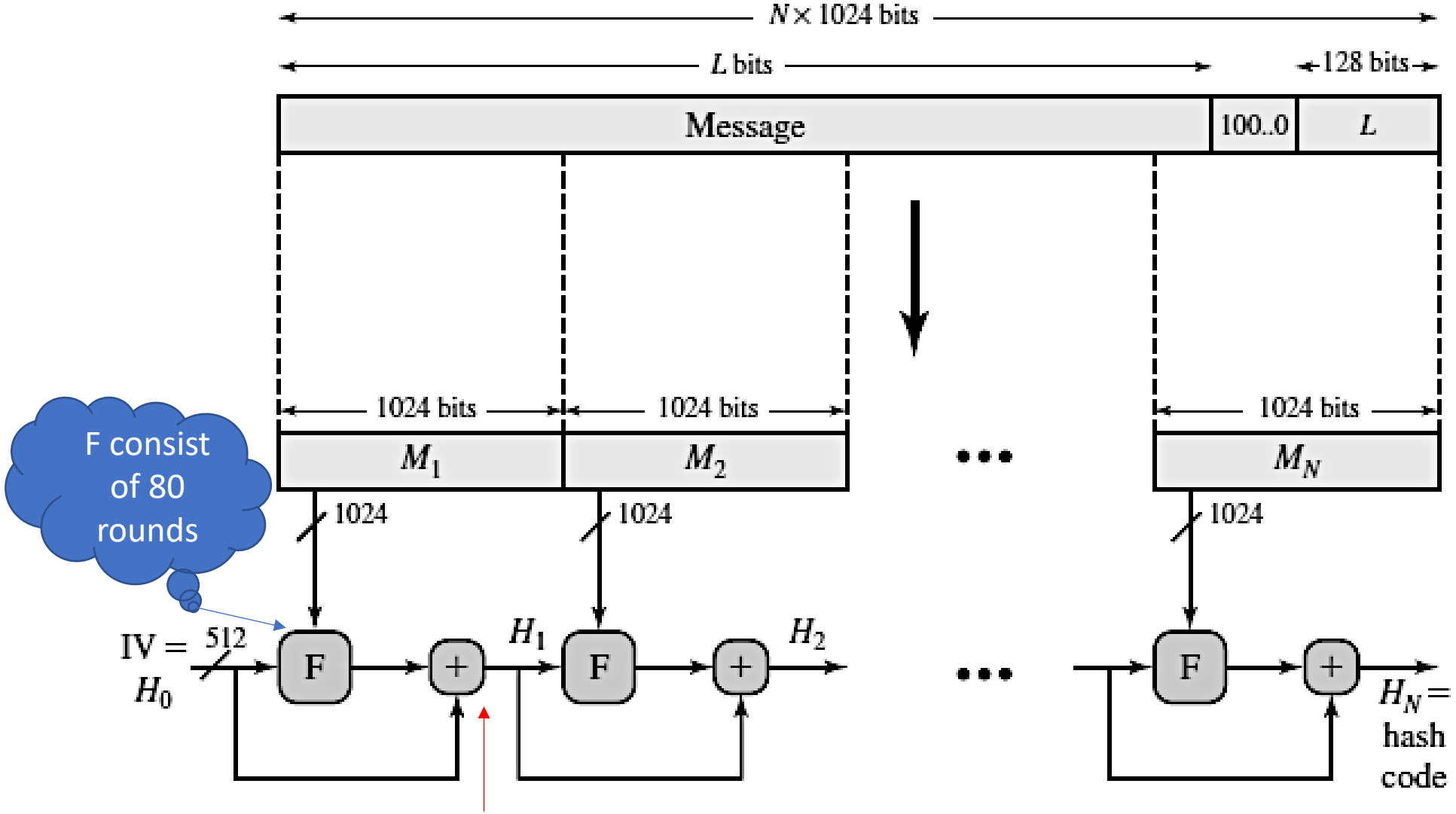
Word-by-word addition mod 2^{64}

- **additive constant K_t**, where $0 \le t \le 79$ indicates one of the 80 rounds

  <u>The constants provide a "randomized" set of 64-bit patterns, which should eliminate any regularities in the input data</u>

  (These words represent the first sixty-four bits of the fractional parts of the cube roots of the first eighty prime numbers)

Hash generation for SHA 512 : processing of all message



P4_hash_SHA

Openssl commands to compute the hash

Using openssl command to calculate hash

```
$ openssl dgst -sha256 file.c
SHA256(file.c)= 0b2a06a29688...(omitted)...1f04ed41d1

$ openssl sha256 file.c
SHA256(file.c)= 0b2a06a29688...(omitted)...1f04ed41d1

$ openssl md5 file.c
MD5(file.c)= 919302e20d3885da126e06ca4cec8e8b

$ openssl dgst -md5 file.c
MD5(file.c)= 919302e20d3885da126e06ca4cec8e8b
```

Application of hash functions

- Integrity verification
- Committing a secret without telling it
- Password verification

Ref. W. Du

Application: Integrity validation

- One-bit change in input, leads to a significant change in hash value
  - try it!

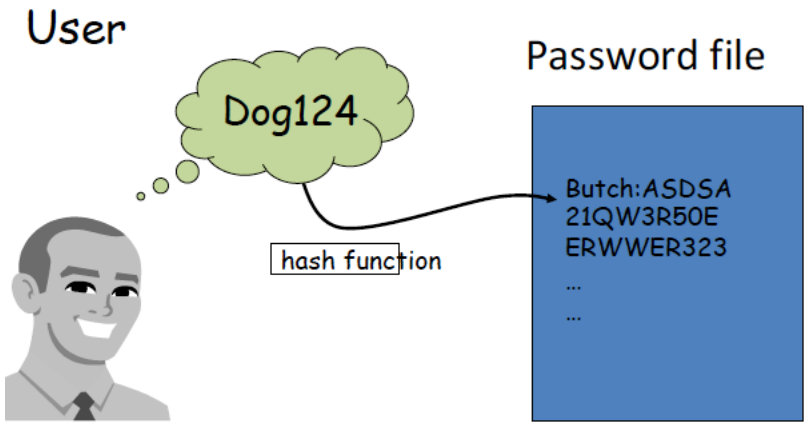
- Example usage:
  - Detect if file downloaded from website is corrupted

Ref. W. Du

Application: Committing to a secret without revealing it

- Example: Have a software that you can sell to predict stock-market
  - To justify it works without exposing the prediction that you want to sell, **publish the hash** of the prediction of next day (secret)
    - **One-way property:** Disclosing the hash does not disclose the original message (secret is kept)
    - **Collision resistant property:** You can’t claim a different message later

Application: Password verification



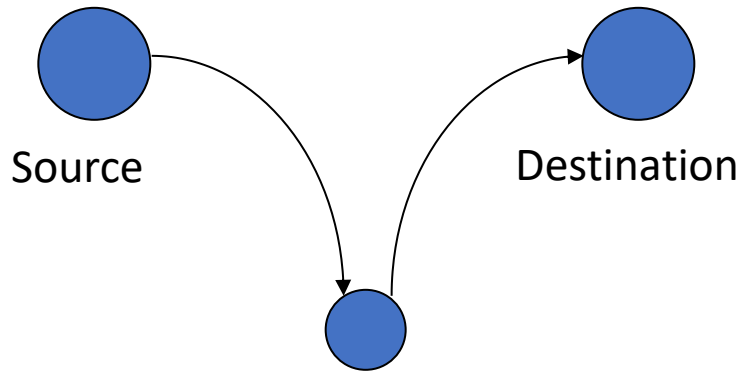
- Using password for authentication is typical
- **Bad practice:** store passwords in plaintext
  - Is system is compromised, passwords will be leaked
- Common practice: store hash of the passwords
  - If the system is compromised and hash is revealed that doesn't reveal passwords (one-way property)

Adding Salt to the Input

- ISSUE: same input , same algorithm ➔ same hash
  - Two users with same password will result in same hash
- **Dictionary Attack**
  - Put candidate words in a dictionary
  - Try each against the targeted password hash to find a match
- Use SALT: Append a random string to input (e.g. password) then get the hash
  - Random string is called salt

Integrity validation requires keyed hash: Message Authentication Code (MAC)

- Communication over a network encounter man in the middle attack (MIMA)
  - Attacker intercept data and modify it



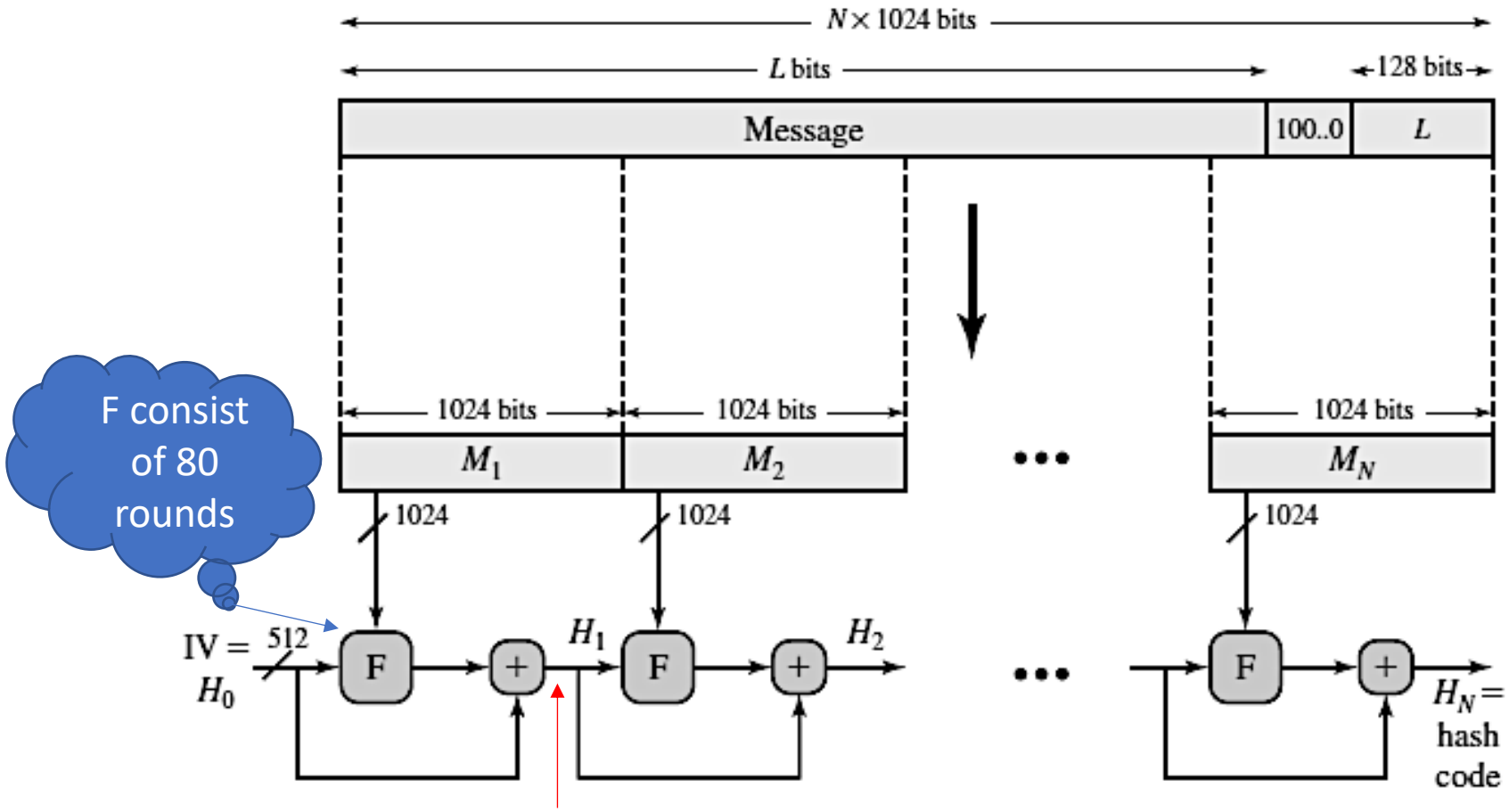
- Attacker can compute the one-way hash and append it
  - Receiver validate the hash which is valid since it is computed based on the modified message

- How to deal with this?

Hash generation for SHA 512 : processing of all message

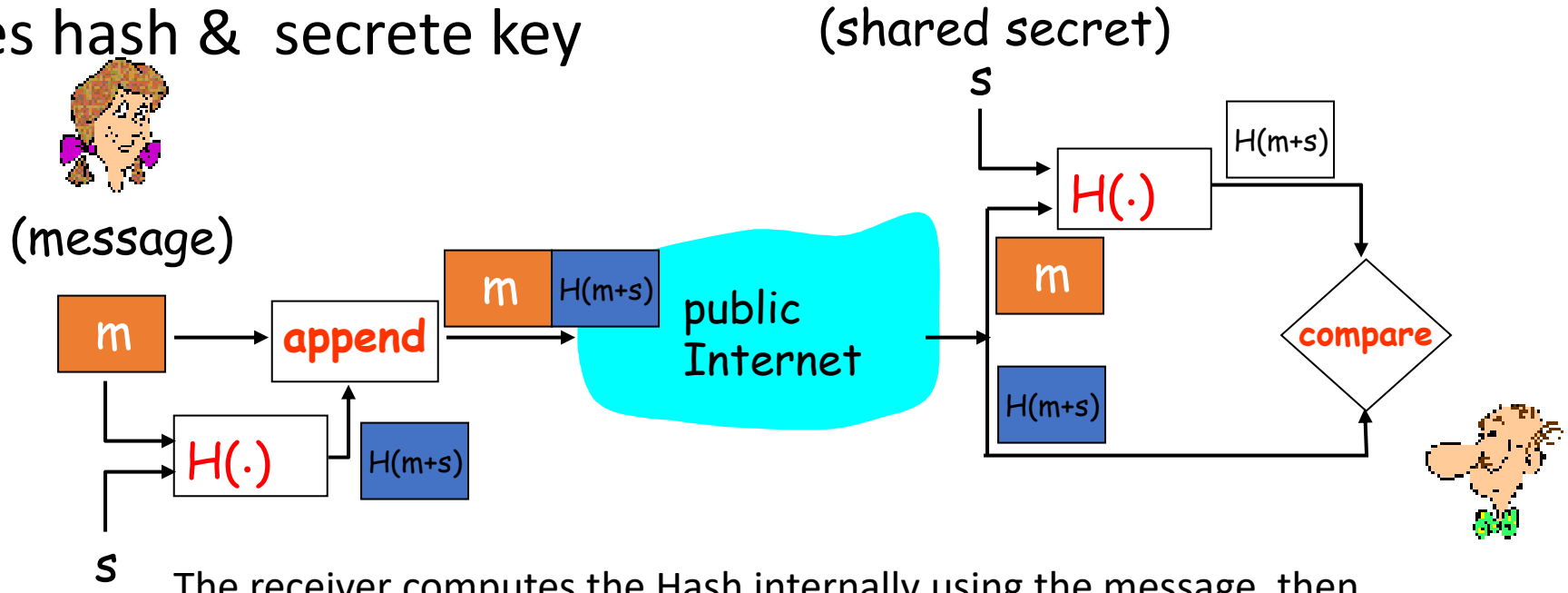
The repetition of the basic function F produces results that are well mixed! it is unlikely that two messages will have the same hash code

L here is input length not hash length



Recall: Message authentication codes (MAC) for integrity validation

- MAC uses hash & secrete key

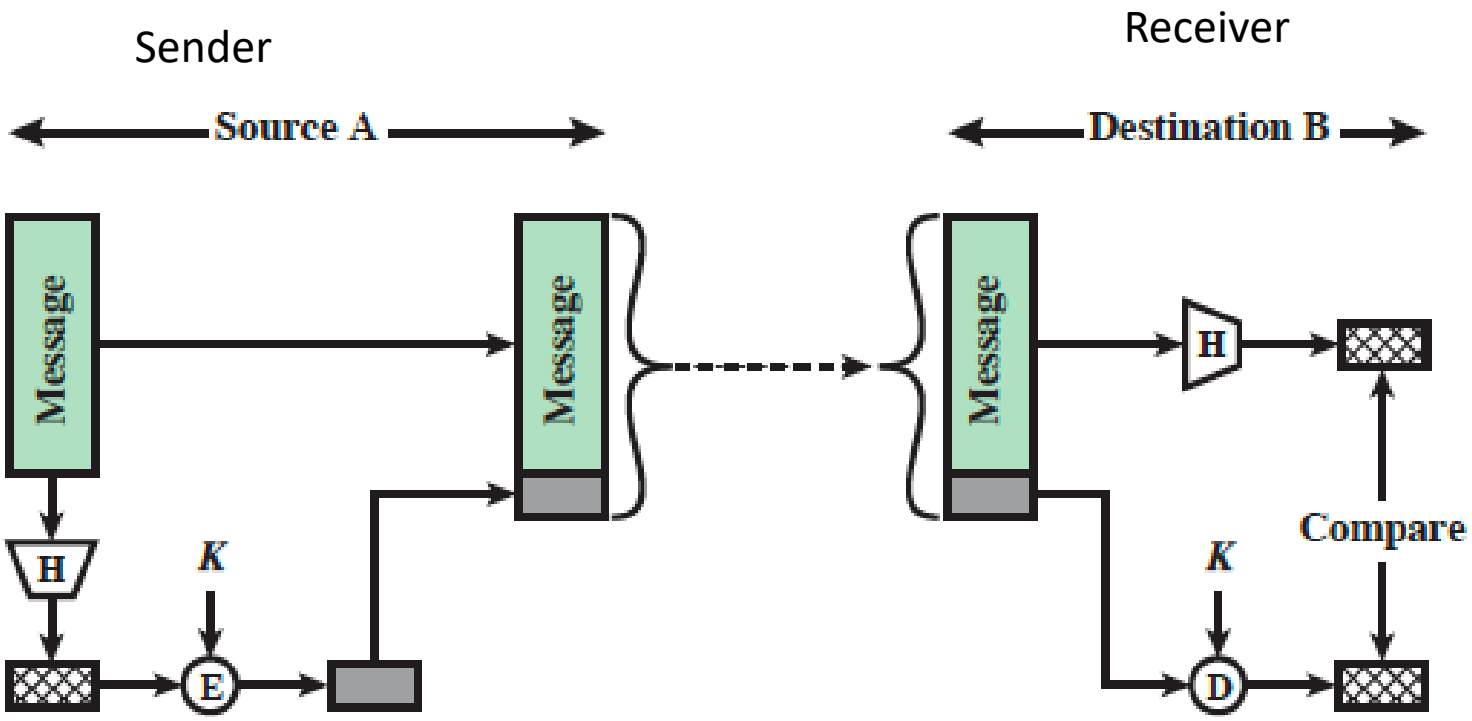


The receiver computes the Hash internally using the message, then compares it to Hash appended to in the transmission.. If they are the same then

- This ensures that message has not been manipulated in transit
- It also ensures that receiver is communicating with the claimed source (since the shared secret key is used to generate the Hash)
  - If a different key is used, the Hash will not be the same
  - Secret key is between every transmitter-receiver pair

Message authentication can also be through using symmetric encryption

- Using the symmetric key, just encrypt hash with for example AES

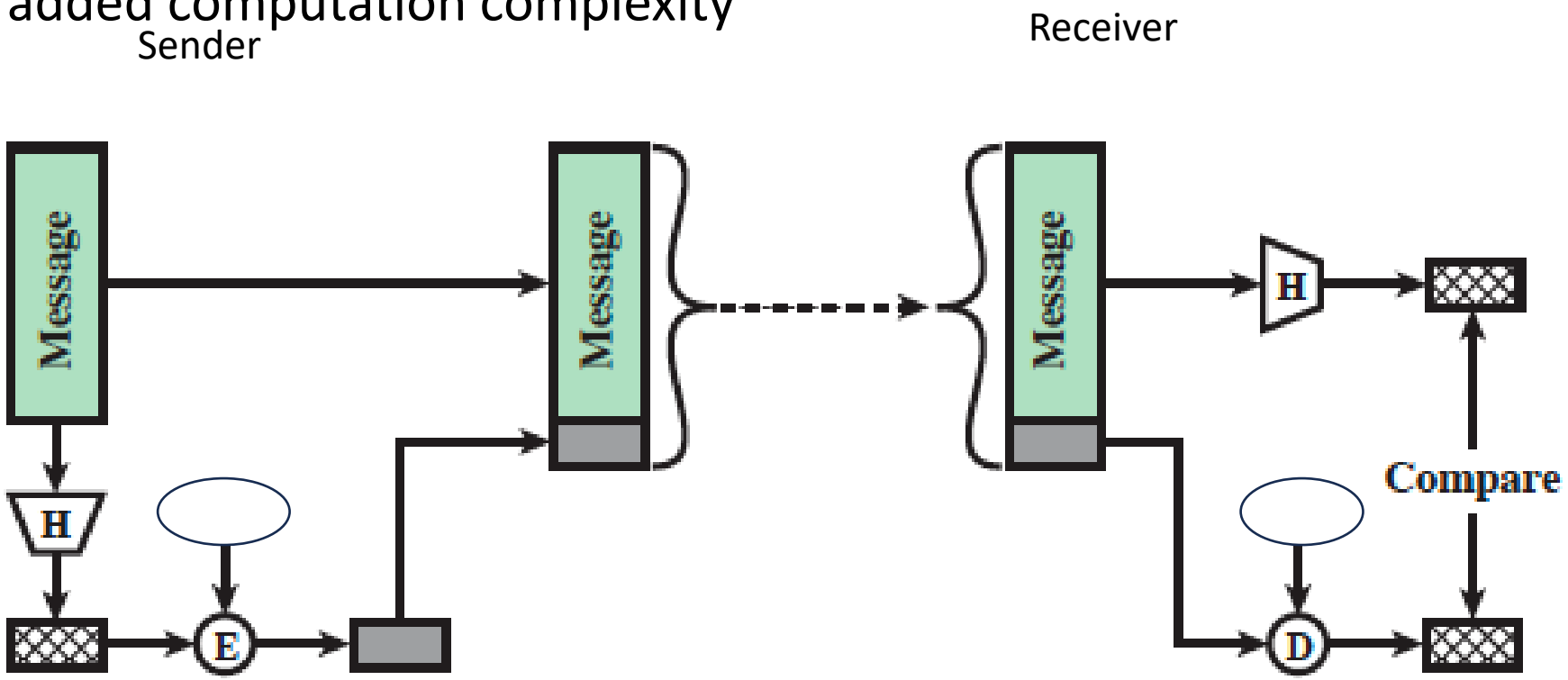


Disadvantage: added computation complexity

Message authentication can also be through using public key encryption

Advantage: Authenticate message and the source

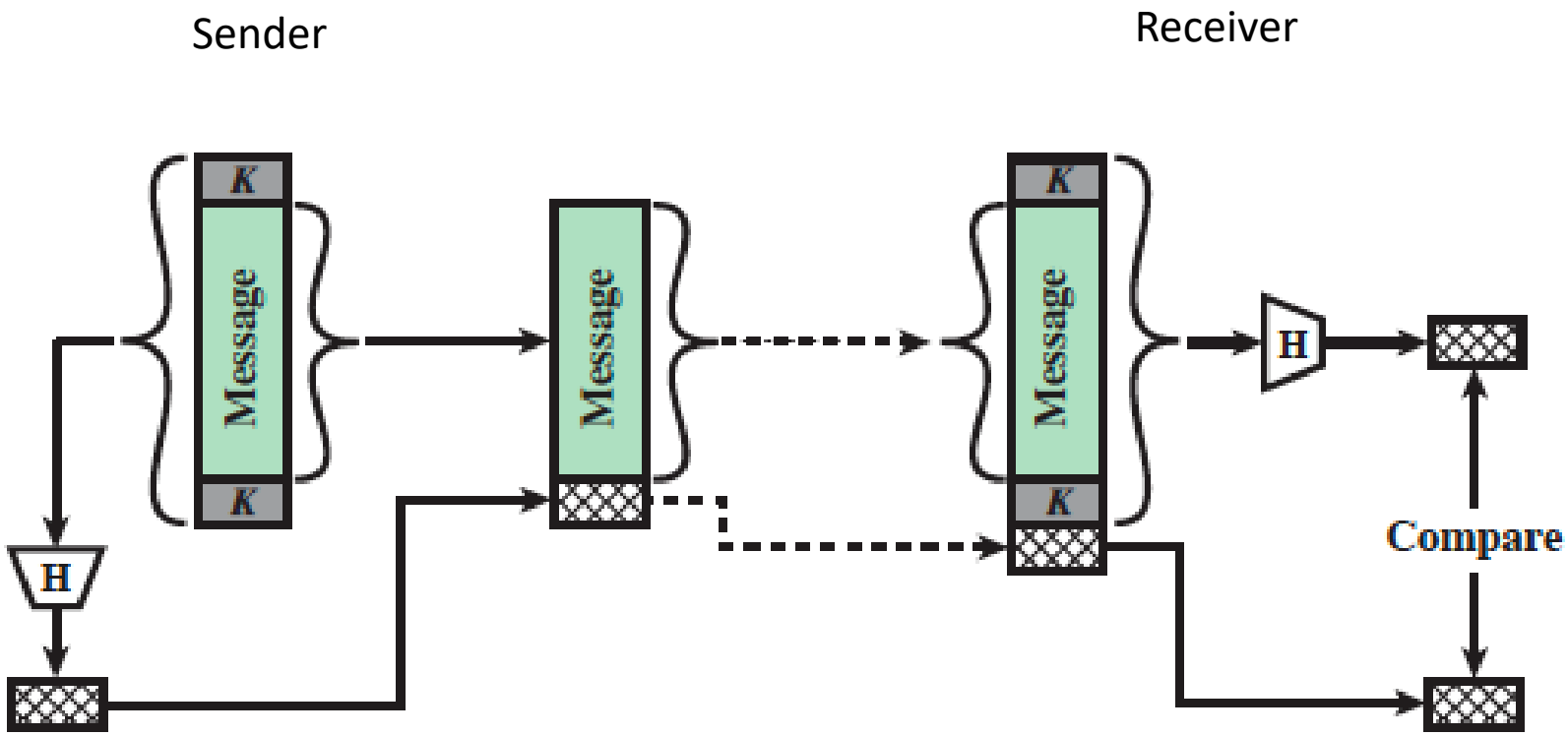
Disadvantage: added computation complexity



Top hat: Q_Hash_Assymetric crypto

Message authentication using HMAC – Common Approach

- Using a secret K and cryptographic hash function



Message Authentication Code (MAC) -- should use a shared secret

- Use a shared **secret (key)** between sender and receiver in the hash

- **MITM** cannot compute hash without secret key

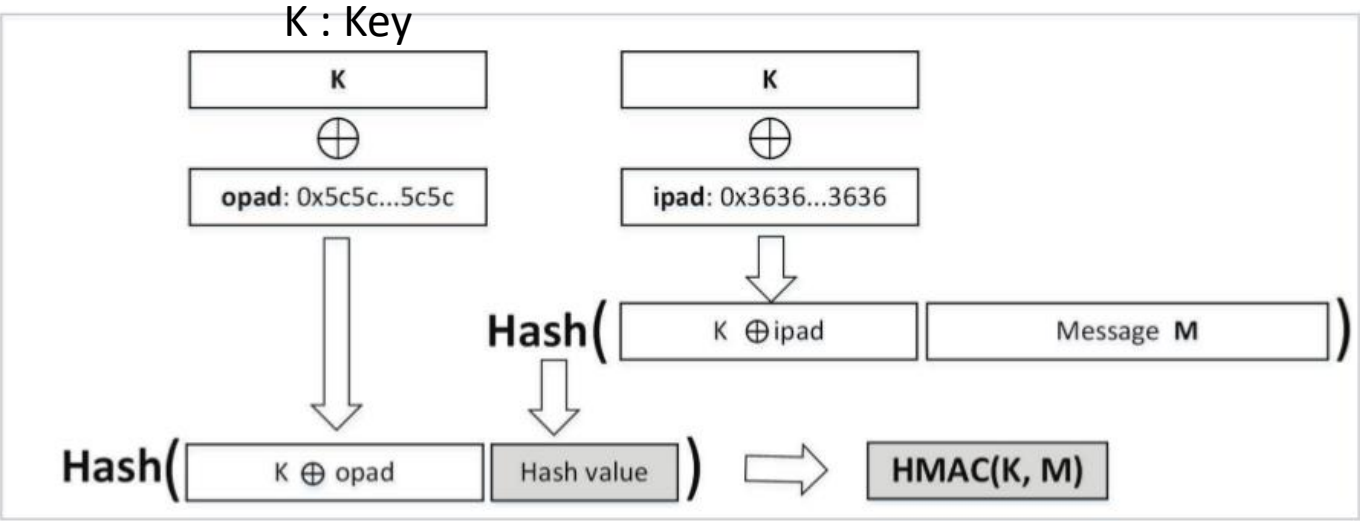
Keyed-Hash Message Authentication Code (HMAC)

- Develop message authentication code (MAC) from cryptographic hash
  - SHA does not rely on a secret key
- Mandatory for IP security and used in TLS

HMAC requirements -- applies a secrete key and well-known hash functions

- **HMAC** use the **exiting hash functions** (SHA) and use **key** in a simple and effective way

  - HMAC treats the hash function as a “black box”



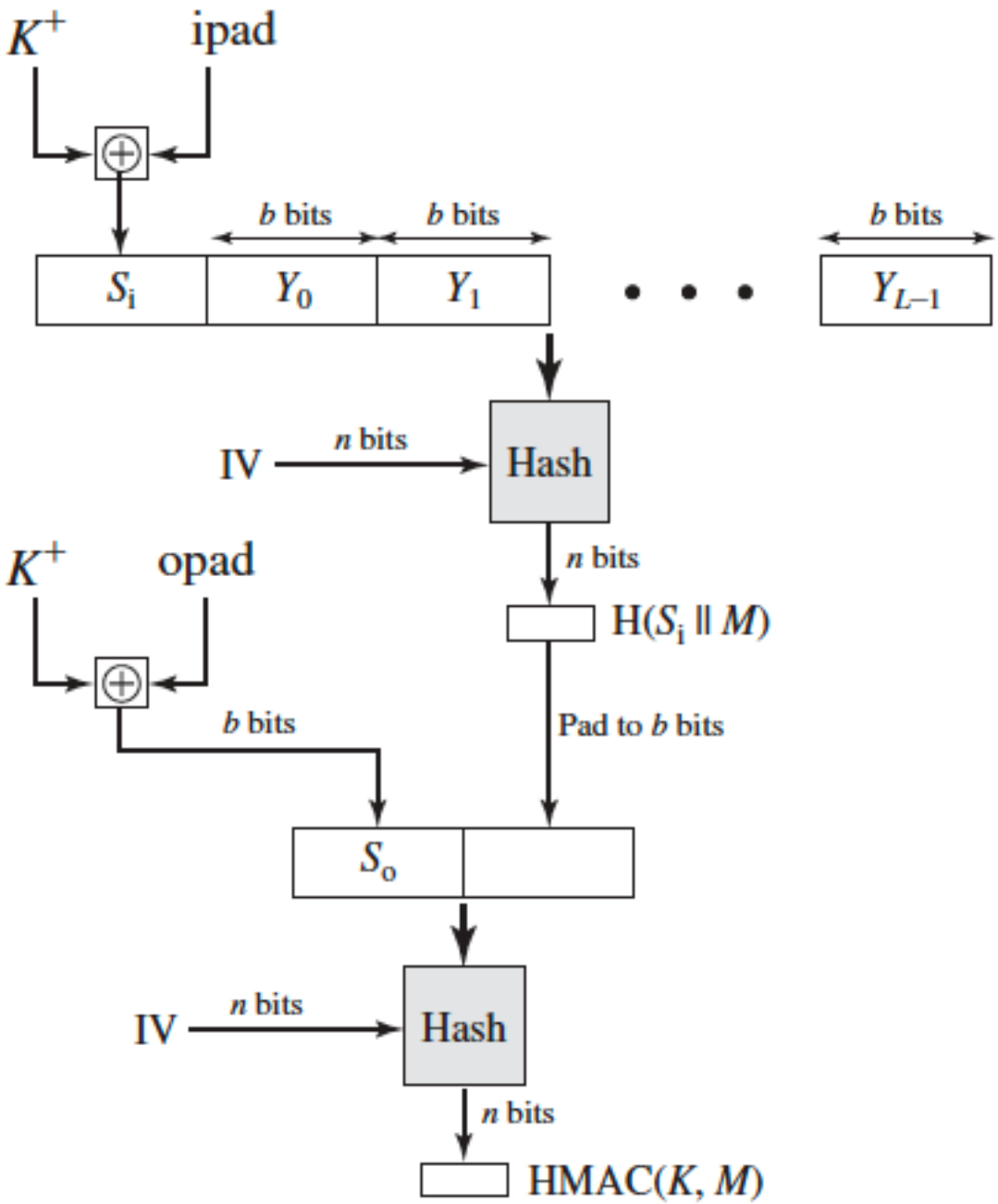
Ref. W. Du. General structure HMAC

Keyed Hash Function: Parameters

H	=	embedded hash function (e.g., SHA)
M	=	message input to HMAC (including the padding specified in the embedded hash function)
Y_i	=	ith block of M, $0 \leq i \leq (L - 1)$
L	=	number of blocks in M
b	=	number of bits in a block
n	=	length of hash code produced by embedded hash function
K	=	secret key; if key length is greater than b, the key is input to the hash function to produce an n-bit key; recommended length is $\geq n$

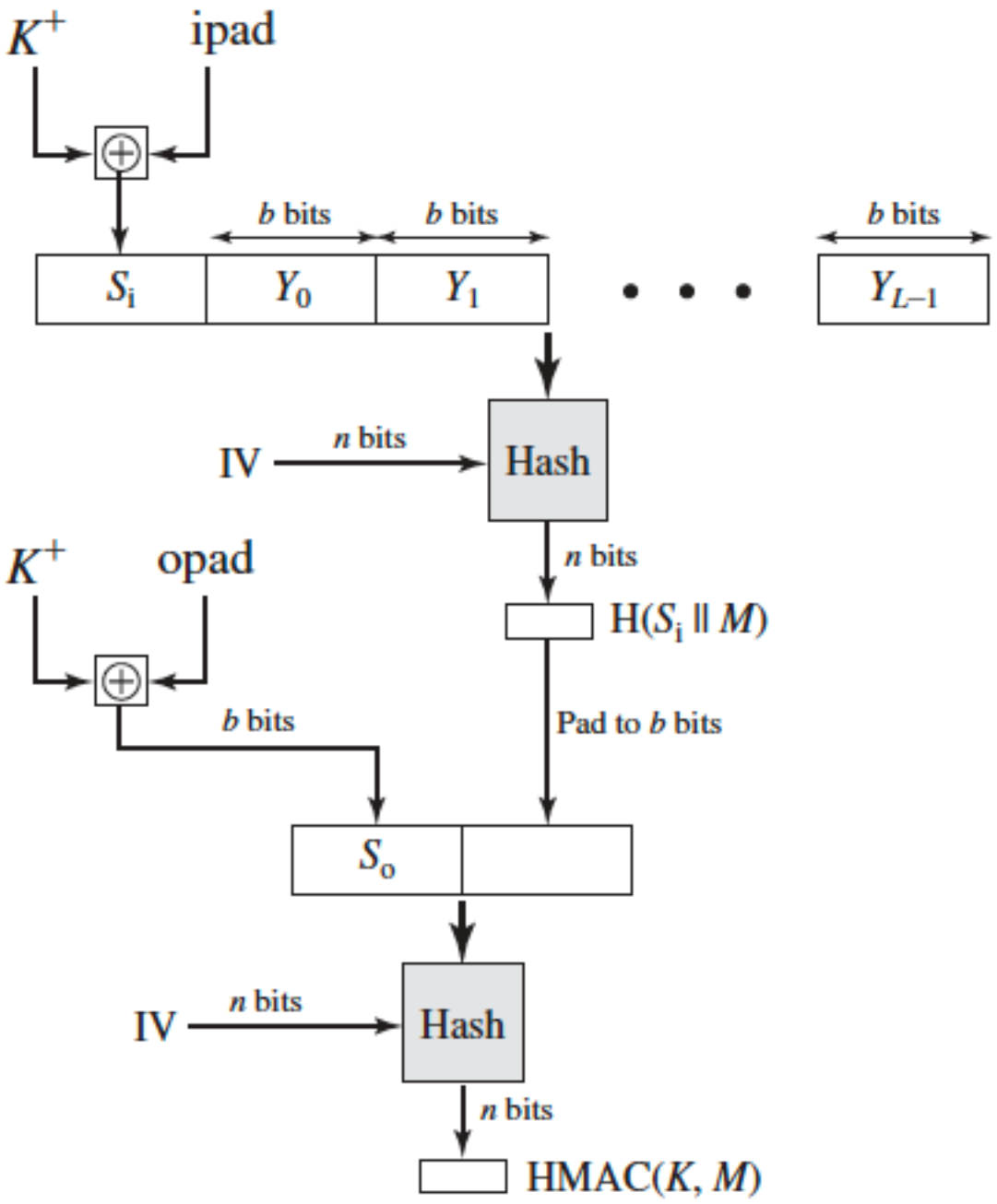
HMAC structure

- K^+ = K padded with zeros on the left to be *b* bits in length
  - e.g., if K 160 bits and b = 512, then K will be appended with 44 zero bytes 0x00
- opad, ipad are fixed constants
  - ipad = 00110110 repeated b/8 times
  - opad = 01011100 repeated b/8 times



HMAC structure

- S_i and S_o ➔pseudo- randomly generated two keys from Key K
  - XOR with ipad results in flipping one-half of the bits of K
  - XOR with opad results in flipping one-half of the bits of K, but a different set of bits



HMAC

1. Append zeros to the left end of K to create a b-bit string K^+

2. XOR (bitwise exclusive-OR) K^+ with ipad to produce the b-bit block

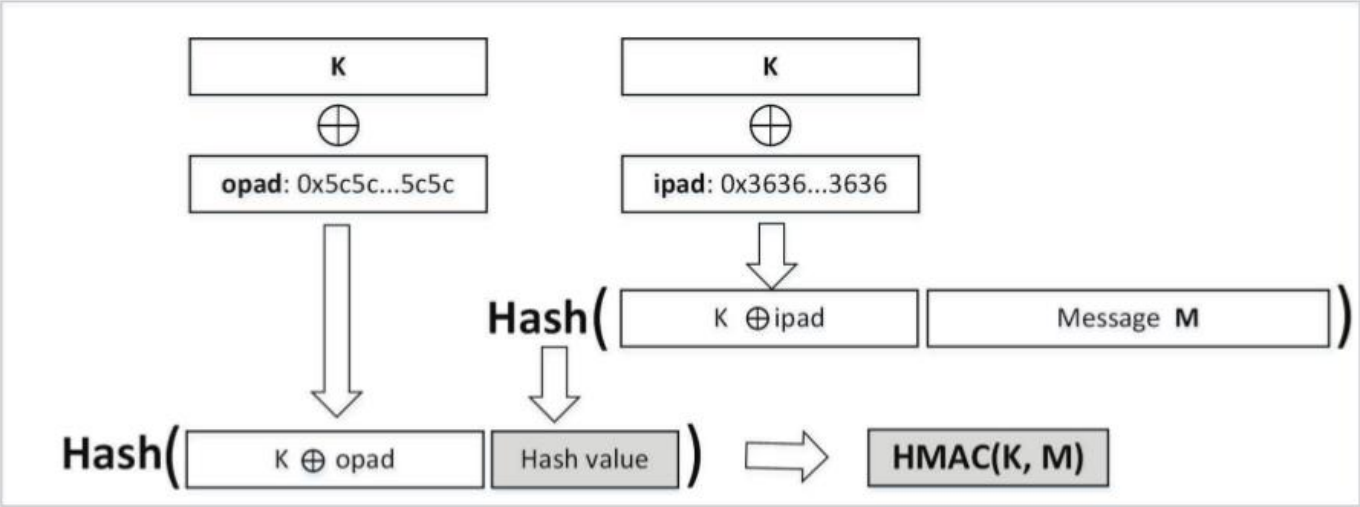
3. Append M to S_i.

4. Apply H to the stream generated in step 3.

5. XOR K^+ with opad to produce the b-bit block S_o.

6. Append the hash result from step 4 to S_o.

7. Apply H to the stream generated in step 6 and output the result.



ipad = 00110110 repeated b/8 times
opad = 01011100 repeated b/8 times

$$\mathrm{HMAC}(K, M) = \mathrm{H}[(K^+ \oplus \mathrm{opad}) \| \mathrm{H}[K^+ \oplus \mathrm{ipad} \| M]]$$

Note

- It is more common to avoid encryption (symmetric of asymmetric in last couple of slides) and use Keyed HASH (HMAC) for message authentication (integrity validation)
  - To avoid computational burden of encryption

Summary

- One-way hash function is **essential building** block in **cryptography**

- Important Properties: fixed size output, **One-way** and **Collision resistant**

- Example applications
  - Password authentication
  - Integrity validation
  - HMAC used to preserve integrity of communication
  - Blockchain